Reproducing and Extending ComPASS: A Compatible PIM Protocol and Scheduling Solution for Processor–PIM Collaboration

Ruchir Sharma
IIT Guwahati
240101080
s.ruchir@iitg.ac.in

Ruchir Sharma
IIT Guwahati
240101081
ruchir.sharma@iitg.ac.in

Purushottam Jha
IIT Guwahati
240101120
j.purushottam@iitg.ac.in

Aryan Gupta
IIT Guwahati
240101018
aryang1018@iitg.ac.in

***Abstract*—Processing-in-memory (PIM) devices accelerate memory-bound kernels such as GEMV, but commercial designs either block host traffic while PIM runs (HBM-PIM mode switching, about 350 ns per round trip) or need PIM-specific controller changes (GDDR6-AiM). ComPASS (Yu et al., MICRO 2025) proposes a compatible command, `PIM-ACT`, a PIM request generator and a throughput balancer that arbitrates between PIM and host requests. We reproduce and extend ComPASS in two cycle-level simulators. In Samsung PIMSimulator we implement `PIM-ACT`, a 35-optype ISA, architecture-aware optimisation (AAO) and PREA-aware scheduling, verified bit-exactly. In Ramulator we add an LPDDR5-6400 model, a 2-entry PIM queue and four schedulers (FR-FCFS, Batch, ST-BLC, AT-BLC) driving 16 out-of-order cores with SPEC CPU2006 traces, plus a proposed elastic balancer, E-BLC. The legacy mode switch measures 351–352 ns and `PIM-ACT` removes it; over the PIM region it gains $+3.25\%$ (GEMV), $+7.16\%$ (ADD), $+2.36\%$ (MUL) and loses 2.75% on ReLU. Under co-execution the balancers cut host slowdown from 4.99% (FR-FCFS) to 3.49–3.79% at constant PIM bandwidth; E-BLC matches ST-BLC but not AT-BLC. Finally, ComPASS+, a set of seven controller improvements, lowers slowdown to 3.44% versus 3.79% for the AT-BLC of the same matrix (-9% relative), a gain comparable to the run-to-run drift of unmodified policies. We state where our numbers fall short of the paper.**

***Index Terms*—processing-in-memory, memory scheduling, PIM-ACT, LPDDR5, Ramulator, PIMSimulator**

I. Introduction

A. Motivation

Memory-bound workloads, notably the GEMV kernels that dominate large-language-model inference, are limited by off-chip bandwidth. Processing-in-memory moves arithmetic next to the DRAM banks and exploits the much larger internal bandwidth. Commercial devices exist for several memory families (UPMEM on DDR4, Samsung HBM-PIM, SK hynix GDDR6-AiM), yet integrating them into a general-purpose system exposes two problems that ComPASS targets [1]:

1) **Protocol compatibility.** HBM-PIM stays JEDEC-compliant by hiding PIM operations behind *mode switches* (SB $\rightarrow$ HAB $\rightarrow$ HAB-PIM); a full round trip costs about 350 ns during which normal requests stall. GDDR6-AiM instead adds a dozen non-standard commands, which forces a dedicated memory subsystem.
2) **Scheduling interference.** PIM streams are throughput-oriented, host cores are latency-sensitive. Plain FR-FCFS or batch scheduling lets one starve the other, causing unpredictable host slowdown or missed PIM deadlines.

B. The ComPASS proposal in brief

ComPASS introduces (i) `PIM-ACT`, an overloaded `ACTIVATE` that opens multiple banks and carries a 6-bit `optype` selecting the PIM operation, so no mode switch is needed; (ii) a PIM request generator inside the memory controller that expands macro requests into micro-requests; (iii) *architecture-aware optimisation* (AAO), which shortens activation timing for register-only operations; and (iv) a *throughput balancer* with a static (ST-BLC) and an adaptive (AT-BLC) variant that alternate PIM and host turns while hiding rank-wide precharge (PREA) latency.

C. What this project does

We reproduce the protocol-level and scheduler-level results of ComPASS in two open simulators and extend them:

- **Part A (Samsung PIMSimulator).** `PIM-ACT` command, 35-optype ISA, multi-bank SIMD FP16 execution, AAO timing, PREA-aware scheduling, and four verified kernels (GEMV 4096×4096, ADD 1M, MUL 2M, ReLU 4M).
- **Part B (Ramulator).** JEDEC-style LPDDR5-6400 model; 16 out-of-order cores with SPEC CPU2006 traces co-executing with a PIM GEMV stream; FR-FCFS, Batch, ST-BLC, AT-BLC and our new E-BLC.
- **Extensions beyond the paper:** cycle-exact decomposition of the mode-switch cost (351–352 ns, six phases), a three-phase decomposition of the benchmark timeline, a root-cause analysis of the ReLU slowdown, a bitwise cross-protocol verifier, an optype-collision diagnostic, runtime-parameterised AAO/PREA knobs, and the E-BLC scheduler.

D. Summary of findings

- The legacy mode-switch round trip measures **351–352 ns** (167–168 ns entry, 184 ns exit); `PIM-ACT` eliminates all 4,864 bus packets it needs.

Project report, Group 3, Indian Institute of Technology Guwahati, 3 November 2026. Reference paper: S. Yu *et al.*, "ComPASS: A Compatible PIM Protocol Architecture and Scheduling Solution for Processor-PIM Collaboration," MICRO 2025.

- Over the PIM compute region, GEMV without AAO gains $+3.25\%$ (paper: $+2.54\%$); other kernels fall short of the paper's gains and ReLU is slower than baseline (-2.75%).
- In co-execution with 16 cores, AT-BLC lowers host slowdown from 4.99% to 3.49% (a 30% relative reduction) at unchanged PIM bandwidth; ST-BLC and E-BLC reach 3.78% and 3.79%.
- Balancers issue about 30% *more* PREA commands than FR-FCFS: the benefit is in hiding their latency, not in reducing their count.
- ComPASS+ (Section IX) reaches 3.44% slowdown against 3.79% for the AT-BLC of its own matrix; the gain is of the size of run-to-run drift (Section IX-F).
- Our N:P parameter sweep is *inconclusive*: the host quota N is not read by the simulator, so every point is identical (Section V-G).

E. Report organisation

Section II reviews PIM and the ComPASS design. Section III describes our implementation and experimental setup. Section IV reports baseline results and Section V the results of the proposed mechanisms. Section VI compares with the paper and lists limitations; Section VII concludes. Reproduction commands are in Section VIII. Finally, Section IX presents our ComPASS+ controller improvements and compares them quantitatively.

II. Background and the ComPASS Design

A. Why mode switching hurts

In HBM-PIM the host drives three modes using JEDEC-legal commands to reserved addresses: *SB* (normal), *HAB* (all banks, registers writable) and *HAB-PIM* (all-bank execution). Entering and leaving PIM mode requires a predefined command sequence, and no normal request is served meanwhile. A 4096×4096 GEMV on four HBM-PIM modules takes about 13 μs [1], so a 350 ns round trip is a few percent of the kernel, and far more for short kernels or when switching is frequent.

B. The `PIM-ACT` command and optype ISA

`PIM-ACT` activates the target row in *all* banks of a rank and simultaneously carries an `optype`. Optypes 0–31 are compute operations and 32–63 are data transfers; we implement the 35 optypes of the paper's Table 1 (Table I). Because `PIM-ACT` reuses the `ACTIVATE` encoding, standard commands and PIM operations interleave at command granularity. A subsequent column command to the same row and optype is a row hit; a different row (or optype) requires a PREA followed by a new `PIM-ACT`.

C. Request generator and AAO

A *PIM request generator* in the controller holds a 32-entry macro-request buffer and a 4 KB channel data buffer and expands macro requests (e.g. one GEMV tile) into per-bank micro-requests, so the host does not send each PIM command. *Architecture-aware optimisation* lets a device shorten t_{RCD}, t_{RAS}, t_{RC} and t_{RP} for `PIM-ACT` operations that only touch registers (GRF/SRF/CRF), skipping bitline sensing and row restoration. The paper does not publish the cycle counts (they are calibrated through the device's SPD); our factors are given in Section III-B.

TABLE I: Optype encoding implemented in `PimCmd.h` (35 optypes, 6-bit field). Transfers are numbered contiguously (32–36), a cosmetic deviation from the paper's numbering (Table X).

Class	Operation	Optype IDs	# optypes
Compute (0–31)	MUL (FP16 element-wise)	0–7	8
	ADD (FP16 element-wise)	8–15	8
	MAD (multiply-add)	16–23	8
	MAC (multiply-accumulate)	24–25	2
	ReLU (threshold variants)	26–29	4
Transfer (32–63)	MOV (host→RF, bank↔GRF)	32–36	5
Reserved	compute 30–31, transfer 37–63	—	—

D. Schedulers

a) FR-FCFS and Batch.: FR-FCFS [7] prefers ready row hits, then the oldest request. Parallelism-aware batch scheduling (PAR-BS) [8] groups requests into batches (here 64), serves older batches first and uses FR-FCFS inside a batch. In both baselines PIM and host requests share one queue.

b) Throughput balancer.: ComPASS keeps PIM requests in a separate shallow queue and alternates turns: up to P PIM commands, then up to N host commands (Figure 2). If the PIM stream meets a row conflict it releases the bus at once, so the PREA latency overlaps useful host work. ST-BLC fixes $P = N = 64$. AT-BLC re-evaluates N every refresh interval t_{REFI} by comparing PIM progress R_{cur} with the rate R_{tgt} needed to finish within the margin-adjusted deadline $T' = 0.9\,T$:

$$R_{\mathrm{cur}} > (1+\lambda)R_{\mathrm{tgt}} \ \wedge \ U_{\mathrm{host}} > 0.5 \\ \Rightarrow N \leftarrow \min(N_{\max},\, N+d), \tag{1}$$

$$R_{\mathrm{cur}} < (1-\lambda)R_{\mathrm{tgt}} \Rightarrow N \leftarrow \max(N_{\min},\, N-d), \tag{2}$$

with $N_{\min} = 16$, $N_{\max} = 128$, $d = 8$. U_{host} is the fraction of the previous quota the host actually consumed (the "50% slot rule").

III. Implementation and Experimental Methodology

A. Toolchain overview

Two open-source cycle-level simulators are used, because the paper's claims live at two different levels:

- **Samsung PIMSimulator** (derived from DRAMSim2 [6]) models the DRAM protocol and the bank-level SIMD FP16 units of HBM-PIM. It lets us execute real kernels and check results bit-exactly. Simulated at 1 ns per cycle with 16 banks per rank (`system_hbm_64ch.ini`).
- **Ramulator** [5] models a full memory system with trace-driven out-of-order cores. It lets us measure interference between 16 host cores and a PIM stream under different schedulers.

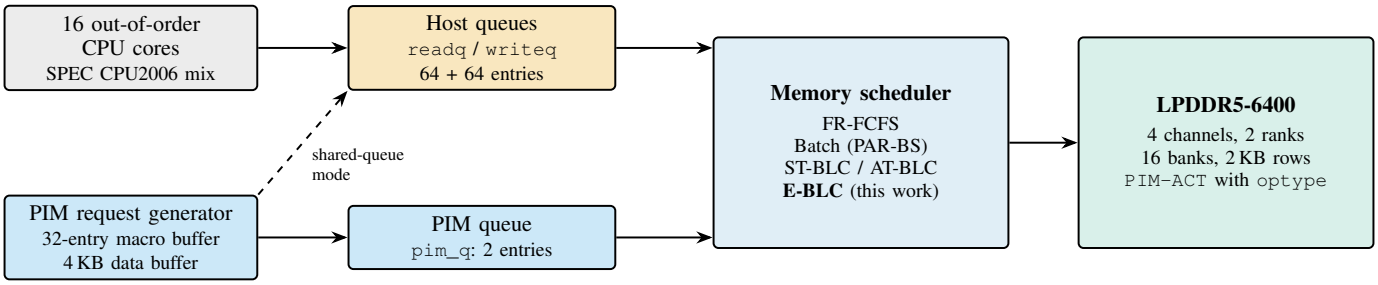


Fig. 1: Memory subsystem modelled in Ramulator. In FR-FCFS and Batch mode PIM micro-requests merge into the shared host queue (dashed); the balancers use the dedicated 2-entry `pim_q`.

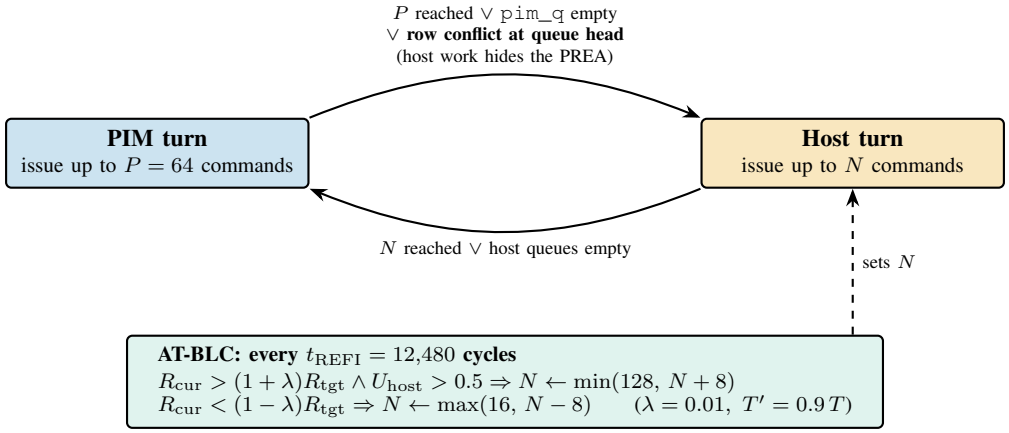


Fig. 2: Turn-switching logic of the throughput balancer. ST-BLC keeps $N = 64$; AT-BLC updates $N \in [16, 128]$ once per refresh interval.

B. Part A: PIMSimulator changes

Command set. `PIM_ACT` added to `BusPacketType`; packets carry `isPIM` and an 8-bit `optype`. We fixed an upstream bug where the 8-argument `BusPacket` constructor omitted `bank(b)` from its initialiser list, leaving uninitialised bank IDs (observed values such as 32707) that triggered assertion failures.

Optype ISA. 35 optypes (Table I); an explicit `NO_OPTYPE = 0xFF` sentinel prevents optype 0 (`OP_MUL_GRF_A_BANK`) from being mistaken for "unset".

Multi-bank execution. `PIMRank::doPIMACT` activates all 16 banks and executes FP16 SIMD operations (MAC for GEMV, ADD, MUL, ReLU) on bank-pair units. `CommandQueue::isIssuable` refuses `PIM-ACT` unless every bank is idle and past its t_{RP}/t_{RRD} recovery; a PRECHARGE issued in `PIM-ACT` mode closes all banks of the rank.

Timing. $t_{RCD,\texttt{PIM-ACT}} = t_{RCD} + 1$ cycle; t_{RCDWR}/t_{RCDRD} enforced separately in `PimActTiming.h`. AAO scales t_{RCD}, t_{RAS}, t_{RC}, t_{RP} by factors 0.5/0.6/0.6/0.5 (floors 2/4/6/2 cycles) for register-only operations. These factors are *our* assumption, exposed as `AAO_TRCD`, `AAO_TRAS`, `AAO_TRC`, `AAO_TRP`.

PREA-aware scheduling. Following the paper's §5.1 principle that same-row, same-optype requests are row hits, `CommandQueue::process_precharge` scans all 16 bank queues before issuing a rank-wide PRECHARGE and suppresses it if another bank still has a pending command on its open row (stopping at barrier packets). Enabled by default; `PIMACT_PREA_AWARE=0` restores naive per-bank precharge. The scan does not compare optypes; a diagnostic (`[B4-HIST]`) confirmed zero same-row optype collisions in all four benchmarks and was validated with an injected positive control.

Verification. Golden FP16 reference data (seed 1113) for the four kernels, plus an *A3* test that runs each kernel under both the legacy and `PIM-ACT` protocols and compares every output burst bitwise: `[A3-OK]`, 0 mismatches. The Google Test suite for the `PIM-ACT` kernels passes 4/4.

C. Part B: Ramulator changes

LPDDR5-6400 model. New `LPDDR5.h/.cpp`: 3200 MHz bus ($t_{CK} = 0.3125$ ns), 16-bit channel, BL16 (32 B per access), 8 Gb die, 16 banks (4 bank groups × 4), 2 KB rows. Timing in cycles: $nRCD$=18, nRP=18, $nRAS$=42, nRC=60, $nRTP$=8, nWR=34, $nFAW$=16, $nRRD$=4, $nRFC$=140, $nREFI$=12480.

Requests and queues. `Request` gains `is_pim`, `optype`, `batch_id` and distinct PIM read/write types. FR-FCFS and Batch place PIM and host requests in the *same* queue; the balancers use host queues of 64 entries each and a dedicated 2-entry `pim_q`. An embedded request generator (32-entry macro buffer, 4 KB data buffer) expands a GEMV stream into micro-requests per rank, bank, row and column chunk.

PREA accounting. Rank-wide precharges issued when service switches between a PIM stream and the host are counted (`prea_count`).

Policies. FR-FCFS; Batch (size 64, older batch first, FR-FCFS inside); ST-BLC ($P{=}N{=}64$, early turn switch on a PIM row conflict); AT-BLC (per-t_{REFI} update of N, 50% slot rule, $\lambda{=}0.01$, $N_{\min,\max}{=}16, 128$, $d{=}8$); and E-BLC below.

Trace parser. CRLF/whitespace trimming added to `Processor.cpp`; upstream `stoul` threw on trailing `\r`.

D. Proposed scheduler: E-BLC

ST-BLC and AT-BLC serialise turns at channel level, so host requests wait even when they touch banks the PIM stream is not using. E-BLC (Elastic Bank-conflict-aware Balancer) adds two rules on top of the turn mechanism (Figure 3):

1) **BLP interleaving.** If the head PIM and host requests map to different bank groups, the host request is a row hit and the PIM request is not, the host request is issued immediately, without ending the PIM turn.
2) **Elastic quota lending.** During a PIM turn, if the PIM request would miss and a host row hit is ready, the PIM turn lends a slot to the host; the host turn's quota grows by the lent slots and is reset at the next switch.

E. Experimental setup

a) Metrics.: Aggregate host IPC is total retired instructions of the 16 cores divided by total CPU cycles. *Host slowdown* is $(\mathrm{IPC}_{\text{host-only}} - \mathrm{IPC}_{\mathrm{co}})/\mathrm{IPC}_{\text{host-only}}$. *Weighted speedup* normalises per-core IPC to the host-only run. *PIM bandwidth* is PIM bytes moved divided by total cycles, at 3.2 GHz. *PREA count* is the number of rank-wide precharges. The *QoS ratio* is $T_{\mathrm{target}}/T_{\mathrm{actual}}$ with $T_{\mathrm{target}} = 5 \times 10^6$ and T_{actual} the total simulated cycles; ≥ 1 means the target is met. In Part A, *gain* = Baseline cycles/PIM-ACT cycles $- 1$, measured both over the PIM compute region and over the whole run (preload + region + readback).

b) Superseded earlier numbers.: Our first Ramulator prototype reported FR-FCFS/Batch slowdowns of 3.57%/3.65%. The final model puts PIM and host in one shared queue and counts PREA turnaround penalties; all results below use the final model, and the earlier numbers should not be compared with them.

IV. Baseline Results

A. Part A: legacy mode-switch protocol (HBM-PIM baseline)

Table III gives the baseline cycle counts of the four paper kernels. These are preserved exactly across all later changes (a regression check in our test matrix).

a) Execution timeline.: The GEMV PIM region lasts 13,483 cycles (13.5 μs), consistent with the paper's "about 13 μs", but it is only 23.3% of the run (Figure 4). This matters for every whole-run comparison below.

b) Mode-switch cost.: We instrumented the memory controller with cycle timestamps (`[MODE-SWITCH]` tags) and analysed them with `analyze_mode_switch.py`. Table IV breaks the round trip into its phases; Figure 5 shows the totals.

B. Part B: host-only and shared-queue baselines

The reference point for all slowdowns is the *host-only* run: 16 cores, no PIM stream, aggregate IPC 14.0967 in 7,985,629 cycles. Adding the PIM GEMV stream under the two shared-queue baselines (FR-FCFS, Batch) gives Table V and Figure 7.

a) Observations.:

- Sharing a queue with the PIM stream costs the host about 5% of its IPC. Batch scheduling changes this by only 0.06 pp (4.93% vs 4.99%): grouping requests does not protect the host from a streaming PIM client.
- PREA commands jump from 2.5 k (host-only) to ≈117 k, almost all on channels 0 and 2 where host and PIM share banks (Figure 7b). Channels 1 and 3 carry only PIM and need 367–370 PREAs each. This is the turnaround penalty the balancers try to hide.
- PIM bandwidth is 3.13 GB/s (0.78 GB/s per channel) for both baselines, with a PIM row-hit rate of 98.2–98.4%, so the PIM stream itself is not suffering from row thrashing; the host pays for the interleaving.
- The QoS ratio is 0.58: with $T_{\mathrm{target}} = 5 \times 10^6$ cycles, no policy, including the baselines, meets the target in this configuration (see limitations, Section VI).

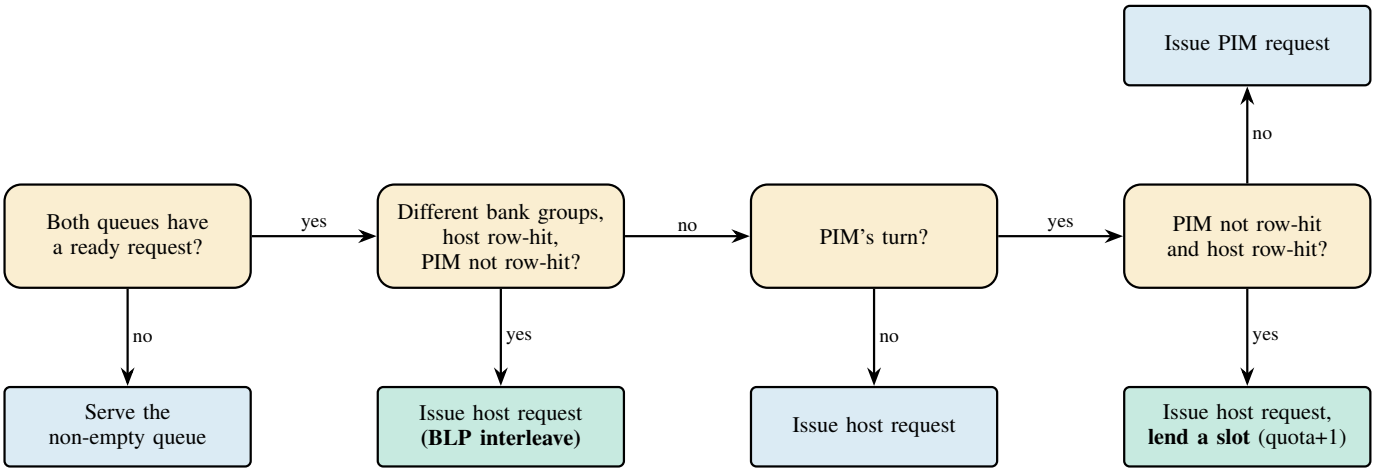
V. Results of the Proposed Mechanisms

A. Part A: `PIM-ACT` protocol

a) Functional correctness.: All four kernels produce bit-identical FP16 results under `PIM-ACT` and the legacy protocol (`[A3-OK]`, 0 mismatches over 4,096 / 1,048,576 / 2,097,152 / 4,194,304 outputs). No activation took the optype-collision path (`[B4-HIST] total=0`).

b) Speed-up versus the baseline protocol.: Table VI and Figure 8 compare PREA-aware `PIM-ACT` (our headline configuration) with the baseline, over the PIM region and over the whole run, alongside the paper's targets.

- **GEMV, AAO off** reproduces the paper's $+2.54\%$ to within 0.71 pp when measured over the PIM region ($+3.25\%$). A back-of-envelope check agrees: removing one 351-cycle round trip from a 13,483-cycle region predicts $+2.67\%$.
- **GEMV, AAO on** reaches $+6.32\%$ against the paper's $+9.65\%$. A runtime histogram (`[AAO-HIST]`) shows that only the 2,048 host-to-register moves (optype 32, about a third of the `PIM-ACT` commands) qualify for the shortened timing; this accounts for the 378-cycle gain. Presumably the remaining activations read bank data and keep full timing (we did not histogram GEMV by optype).
- **ADD, MUL** gain $+7.16\%$ and $+2.36\%$, below the paper's roughly 10–11%. For these kernels the AAO branch never fired (`[AAO-HIST] total=0`): every operation reads a bank and needs full bitline timing, so AAO on/off are identical. We do not know whether the paper's AAO also covers bank-sourced operations.
- **ReLU** is slower than baseline, the opposite of the paper's $+4.5\%$. Section V-C traces this to host readback.
- Whole-run gains are 0.45–1.67% for the three positive kernels because 44–77% of each run is host preload outside the



Turn bookkeeping after each issue: PIM → host after P PIM commands or an empty `pim_q`; host → PIM after N + lent host commands or an empty host queue (lent slots are then reset).

Fig. 3: Decision flow of the proposed E-BLC scheduler for one controller cycle. The two green boxes are the additions to the turn-based balancer.

TABLE II: Simulated system and workload configuration used for the Ramulator experiments.

Component	Setting
Host	16 out-of-order cores, one SPEC CPU2006 trace each: mcf, lbm, milc, leslie3d, soplex, GemsFDTD, libquantum, omnetpp, bzip2, zeusmp, gromacs, cactusADM, hmmer, sjeng, h264ref, xalancbmk. No cache model; 200 k warm-up and 2 M instructions per core limit.
Memory	LPDDR5-6400, 4 channels, 2 ranks/channel, 16 banks/rank, 2 KB row, BL16, timing as in Section III.
Controller queues	`readq=writeq`=64; `pim_q`=2 (balancers); macro buffer 32; data buffer 4 KB.
PIM workload	GEMV micro-request stream, 65,536 micro-ops per channel (262,144 total, 32 B each, 8 MiB).
Balancer	$P = 64$; $N_0 = 64$; $N \in [16, 128]$; $d = 8$; $\lambda = 0.01$; $T = 5 \times 10^6$ cycles, $T' = 0.9\,T$.
Policies	Host-only, FR-FCFS, Batch (64), ST-BLC, AT-BLC, E-BLC.

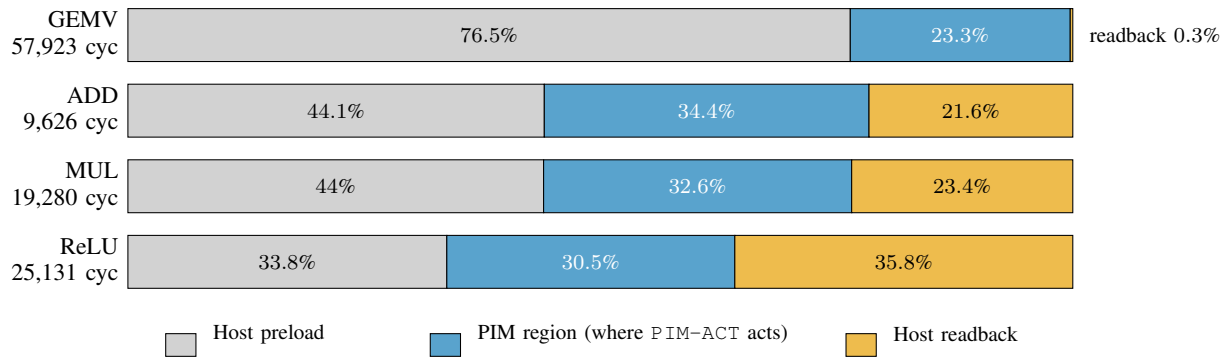


Fig. 4: Share of each baseline run spent in host preload, the PIM compute region and host readback (drawn to scale). `PIM-ACT` can only change the middle segment.

`PIM-ACT` region. Our earlier whole-run-only measurement (+0.62% for GEMV) is what first looked like a failed reproduction.

B. PREA-aware scheduling and phase breakdown

Naive per-bank precharging closes rows that other banks still need; every kernel then pays for repeated `PIM-ACT` activations and loses 16–19% over the whole run. PREA-aware scheduling avoids this (Figures 9 and 10, Table VII).

C. Root cause of the ReLU slowdown

Under PREA-aware `PIM-ACT`, ReLU needs 678 more cycles than the baseline (25,809 vs 25,131). Decomposing by phase:

TABLE III: Baseline (legacy mode-switch) protocol: cycle counts per execution phase (1 cycle = 1 ns).

Kernel	Preload	PIM region	Readback	Total	Preload share
GEMV (4K×4K)	44,283	13,483	157	57,923	76.5%
ADD (1M)	4,242	3,308	2,076	9,626	44.1%
MUL (2M)	8,482	6,286	4,512	19,280	44.0%
ReLU (4M)	8,482	7,660	8,989	25,131	33.8%

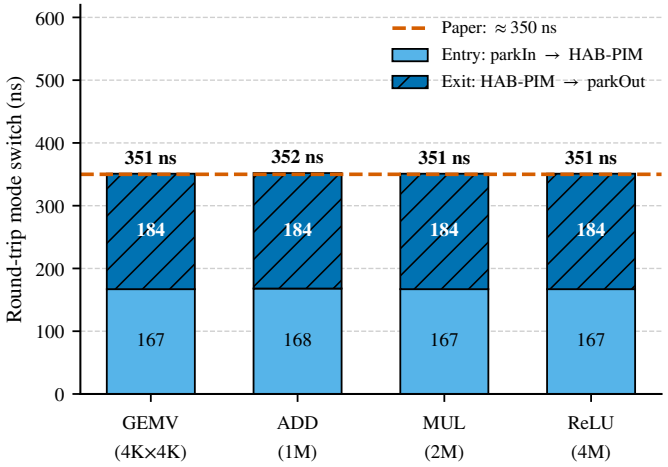


Fig. 5: Measured round-trip mode-switch cost of the baseline protocol for the four kernels, split into entry and exit; the dashed line is the paper's estimate.

Phase	Extra cycles	Share
Host readback	428	63.1%
PIM region	217	32.0%
Preload / pipeline drain	33	4.9%
Total	**678**	100%

Readback dominates: PREA-aware scheduling deliberately keeps rows open across the rank, so when the host then reads the results it hits open rows belonging to other data and incurs 896 extra bank-conflict `ACT`s (64,384 vs 63,488). In the PIM region itself, the baseline alternates activations across even/odd bank pairs so that one pair's activation overlaps the other's compute, whereas rank-wide `PIM-ACT` opens all 16 banks together and cannot pipeline this way. Both effects are inherent to the model we implemented; the paper does not report ReLU in enough detail to tell whether its `PIM-ACT` variant avoids them.

D. Part B: scheduling results

Table VIII and Figure 11 give the main co-execution results for all six configurations.

a) Host protection.: Relative to FR-FCFS, AT-BLC raises aggregate IPC by 1.59% and cuts slowdown by 30% (4.99% → 3.49%); ST-BLC gives +1.27% IPC (24% lower slowdown) and E-BLC +1.26% (24%). Batch is indistinguishable from FR-FCFS. This is the paper's qualitative claim: dedicating a shallow PIM queue and alternating turns bounds host damage.

b) PIM throughput.: PIM bandwidth stays within 3.12–3.16 GB/s for all policies (spread $\leq$ 1.3%), so protecting the host did not cost PIM throughput. Because every policy serves the same 262,144 micro-ops at the same rate, "PIM speedup" relative to FR-FCFS is 1.00–1.01×, i.e. no meaningful difference.

c) PREA commands rise.: The balancers issue 29–31% *more* PREA commands than FR-FCFS (Figure 12a). This is consistent with switching turns on every PIM row conflict, which ends PIM bursts earlier and so creates more PIM-to-host transitions. The gain is therefore not fewer precharges but precharge latency overlapped with host work, which shows up as lower host slowdown. Our figure is labelled "PREA command count versus FR-FCFS" because we cannot claim "PREA elimination".

d) QoS ratio and run length.: The ratio $T_{\text{target}}/T_{\text{actual}}$ is 0.58–0.59 for every policy (Figure 12b). Since T_{actual} is total simulated cycles, which also depend on when the slowest host core finishes, it is not a clean PIM-completion metric. ST-BLC finishes first (8.489 M cycles, 1.0% shorter than FR-FCFS); AT-BLC has the best IPC yet the longest run (8.593 M), because IPC and run length weigh instructions retired by faster cores differently.

E. Adaptive quota behaviour of AT-BLC

Figure 13 shows $N(t)$ for channel 0 (all four channels logged identical timelines). It starts at 56, drops by 8 per epoch to $N_{\min} = 16$ within 0.08 M cycles, as PIM is initially behind its target rate, stays there for ≈0.7 M cycles, then climbs to $N_{\max} = 128$ by 0.96 M cycles when PIM runs ahead and the host used more than half its previous quota. It holds 128 for ≈1.8 M cycles, falls back to 16 around 2.9 M, and remains there to the last logged epoch (4.9 M). Of the 392 logged epochs, 219 sit at 16 and 142 at 128; only 31 are transitional. AT-BLC thus behaves as a slow bang-bang controller with long plateaus, not a smoothly tracking one.

F. Proposed E-BLC

E-BLC lands at 3.79% slowdown, essentially equal to ST-BLC (3.78%), 0.30 pp behind AT-BLC, and at 3.14 GB/s PIM bandwidth; it issues 152.0 k PREAs. We therefore *cannot* claim that BLP interleaving and quota lending improve on the paper's balancers in this setup. Plausible reasons, which we have not tested: (i) host traffic occupies only two of four channels, limiting concurrent cross-bank-group opportunities; (ii) the GEMV stream hits 98% row-hit rate, so PIM rarely conflicts and there is little quota to lend; (iii) E-BLC has no adaptive N. Note that an earlier plotting script in the repository contains hard-coded placeholder numbers that show E-BLC at 2.92% slowdown; they are used only if the CSV is missing and were *not* measured. All numbers in this report come from `compass_evaluation_metrics.csv`.

G. N:P parameter sweep (inconclusive)

We attempted to mirror the paper's sensitivity study by sweeping the requested host quota $N \in \{16, 32, 64, 96, 128\}$ with $P = 64$ for ST-BLC, AT-BLC and E-BLC. Figure 14 shows every curve flat: results are identical for all N. The reason is in our code, not the architecture: the controller initialises

TABLE IV: Measured phases of the legacy mode-switch round trip (GEMV window; ADD differs by ≤ 1 cycle). Bus packets: 4,864 in total.

#	Phase	Cycles	Packets	Function
1	`parkIn()`	87	2048	row-parking reads across all 16 banks
2	SB $\rightarrow$ HAB	41	256	mode-register writes (`PIM_ABMR_RA`)
3	HAB $\rightarrow$ HAB-PIM	11	128	reset PIM program counter
4	PIM compute	13,132	—	arithmetic execution
5	HAB-PIM $\rightarrow$ HAB	11	128	write to `pim_reg_ra_`
6	HAB $\rightarrow$ SB	22	256	mode-register writes (`PIM_SBMR_RA`)
7	`parkOut()`	86	2048	row-parking reads across all 16 banks
	Entry overhead (1–3, incl. gaps)	167		GEMV; ADD 168
	Exit overhead (5–7, incl. gaps)	184		all kernels
	Round trip	**351**		ADD 352 ns; paper: "about 350 ns"

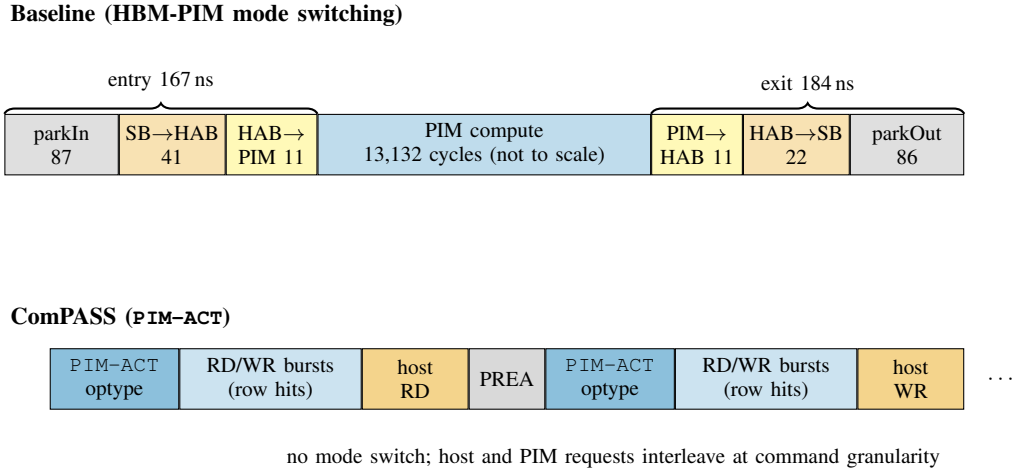


Fig. 6: Command timelines. The baseline protocol stalls normal traffic for the 351 ns entry and exit sequences; `PIM-ACT` removes them.

TABLE V: Ramulator baselines: host-only, FR-FCFS and Batch (PAR-BS, batch 64), 16 cores + 262,144 PIM micro-ops.

Policy	Host IPC	Slowdown (%)	WS	PIM BW (GB/s)	PREA ($\times 10^3$)	QoS ratio	Cycles ($\times 10^6$)
Host-only	14.0967	0 (ref.)	1.0000	—	2.5	—	7.986
FR-FCFS	13.3931	4.99	0.9470	3.130	116.7	0.583	8.577
Batch	13.4018	4.93	0.9478	3.126	117.0	0.582	8.588

TABLE VI: `PIM-ACT` versus the baseline protocol (cycles; gain = baseline/`PIM-ACT` -1). "Paper" values are those stated in the paper's text (§6, Figure 7).

	Base	AAO off		AAO on		Paper		Naive PREA	
Kernel	total	total	region	total	region	off	on	total	region
GEMV	57,923	57,567	+3.25	57,189	+6.32	+2.54	+9.65	71,033	−49.1
ADD	9,626	9,468	+7.16	9,468	+7.16	+9.68	+10.00	11,602	−31.3
MUL	19,280	19,194	+2.36	19,194	+2.36	+11.38	+11.53	23,080	−37.2
ReLU	25,131	25,809	−2.75	25,809	−2.75	+4.51	+4.65	30,903	−42.7
Whole-run gain (%): GEMV +0.62 / +1.28 (AAO off / on); ADD +1.67; MUL +0.45; ReLU −2.63.									

`N_limit` to 64 and never reads N from the configuration file, and `parameter_sweep.py` accepts `n_limit` but never writes it to the generated config. The sweep also used a shortened setup (1 M instructions, 32,768 PIM micro-ops, $T = 3 \times 10^6$), so its absolute values (IPC $\approx$13.1, $\approx$4.2 M cycles) are not comparable with Table VIII. We report the figure as a negative result and list the fix under future work; no sensitivity conclusion should be drawn from it.

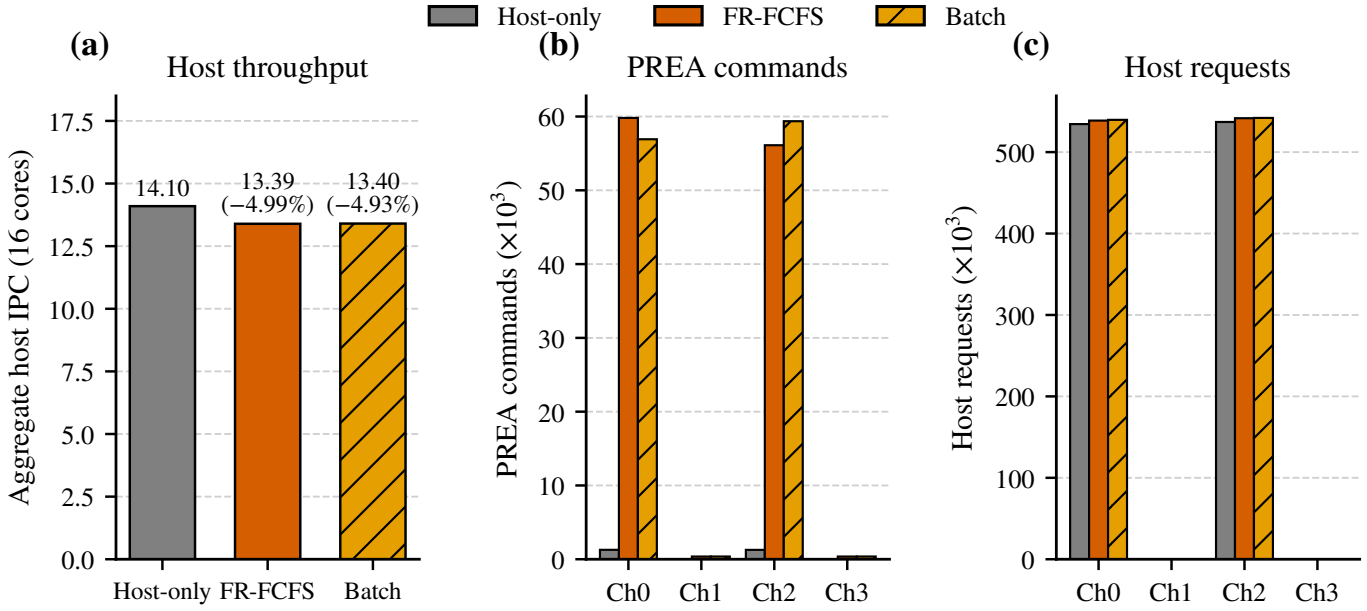


Fig. 7: Baseline results. (a) Aggregate host IPC of the 16 cores; (b) rank-wide PREA commands per channel; (c) host requests per channel. Host traffic reaches only channels 0 and 2 (address mapping of the trace set), while PIM spreads evenly over all four.

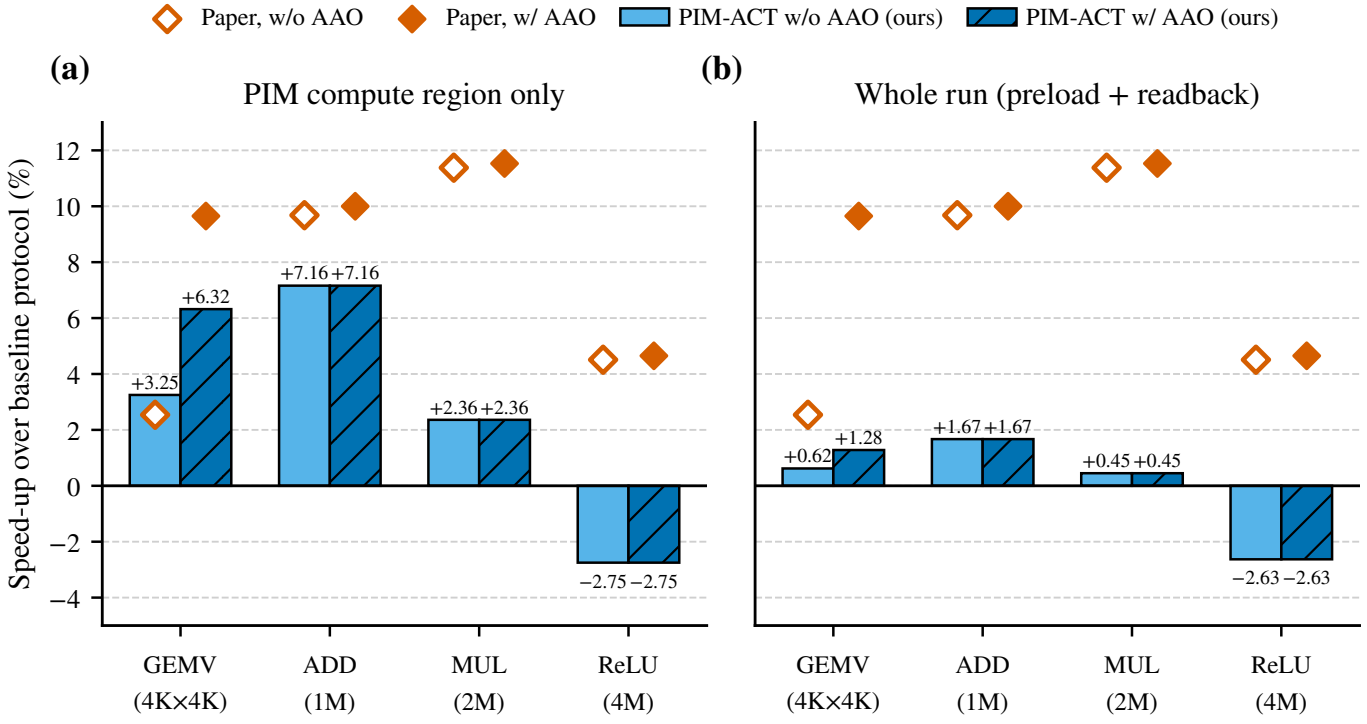


Fig. 8: Reproduction of the paper's Figure 7 speed-ups: (a) over the PIM compute region only, (b) over the whole run. Bars are ours; diamonds are the paper's targets.

VI. Discussion

A. Comparison with the paper

B. Deviations and limitations

C. Beyond the paper

Four analyses go beyond what the paper reports: (i) the six-phase decomposition of the mode-switch round trip with cycle

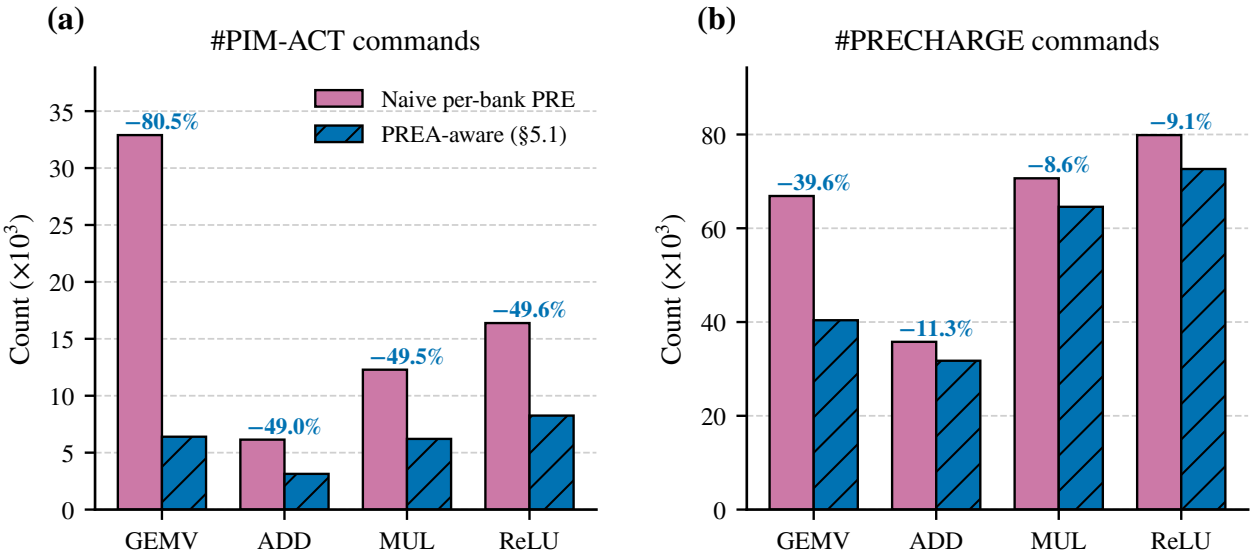


Fig. 9: Effect of PREA-aware scheduling on (a) `PIM-ACT` and (b) PRECHARGE command counts relative to naive per-bank precharge.

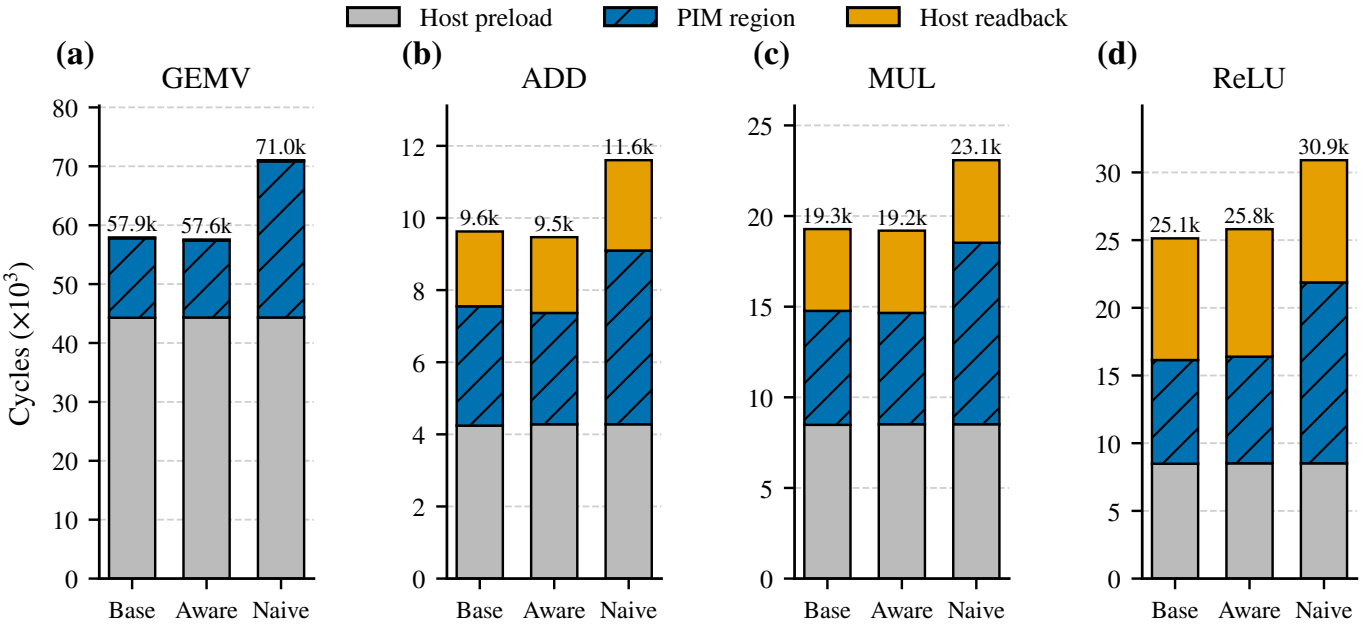


Fig. 10: Cycle breakdown per kernel: baseline, PREA-aware `PIM-ACT` and naive-precharge `PIM-ACT` (labels give total cycles). Relative to the baseline, the naive variant inflates the PIM region by 1.5–2.0×.

TABLE VII: Command counts under naive per-bank precharge and PREA-aware scheduling (AAO off).

	#PIM-ACT			#PRECHARGE		
Kernel	naive	aware	change	naive	aware	change
GEMV	32,896	6,400	−80.5%	66,880	40,384	−39.6%
ADD	6,144	3,136	−49.0%	35,776	31,744	−11.3%
MUL	12,288	6,208	−49.5%	70,656	64,576	−8.6%
ReLU	16,384	8,256	−49.6%	79,872	72,640	−9.1%

timestamps; (ii) the preload/region/readback decomposition that explains why whole-run speed-ups look small; (iii) the ReLU root-cause attribution (63% readback conflicts, 32% bank serialisation); and (iv) the optype-collision diagnostic with a positive control. The E-BLC scheduler is our design contribution, but, as stated above, its measured benefit is nil in this configuration.

TABLE VIII: Co-execution results: 16 cores + PIM GEMV stream on 4-channel LPDDR5-6400. Bold marks the best co-execution value in a column.

Policy	Host IPC	Slowdown (%)	WS	PIM BW (GB/s)	PREA ($\times 10^3$)	QoS ratio	Cycles ($\times 10^6$)
Host-only	14.0967	0 (ref.)	1.0000	—	2.5	—	7.986
FR-FCFS	13.3931	4.99	0.9470	3.130	**116.7**	0.583	8.577
Batch	13.4018	4.93	0.9478	3.126	117.0	0.582	8.588
ST-BLC	13.5636	3.78	0.9541	**3.162**	152.7	**0.589**	**8.489**
AT-BLC	**13.6055**	**3.49**	**0.9565**	3.124	151.0	0.582	8.593
E-BLC (ours)	13.5623	3.79	0.9513	3.140	152.0	0.585	8.548

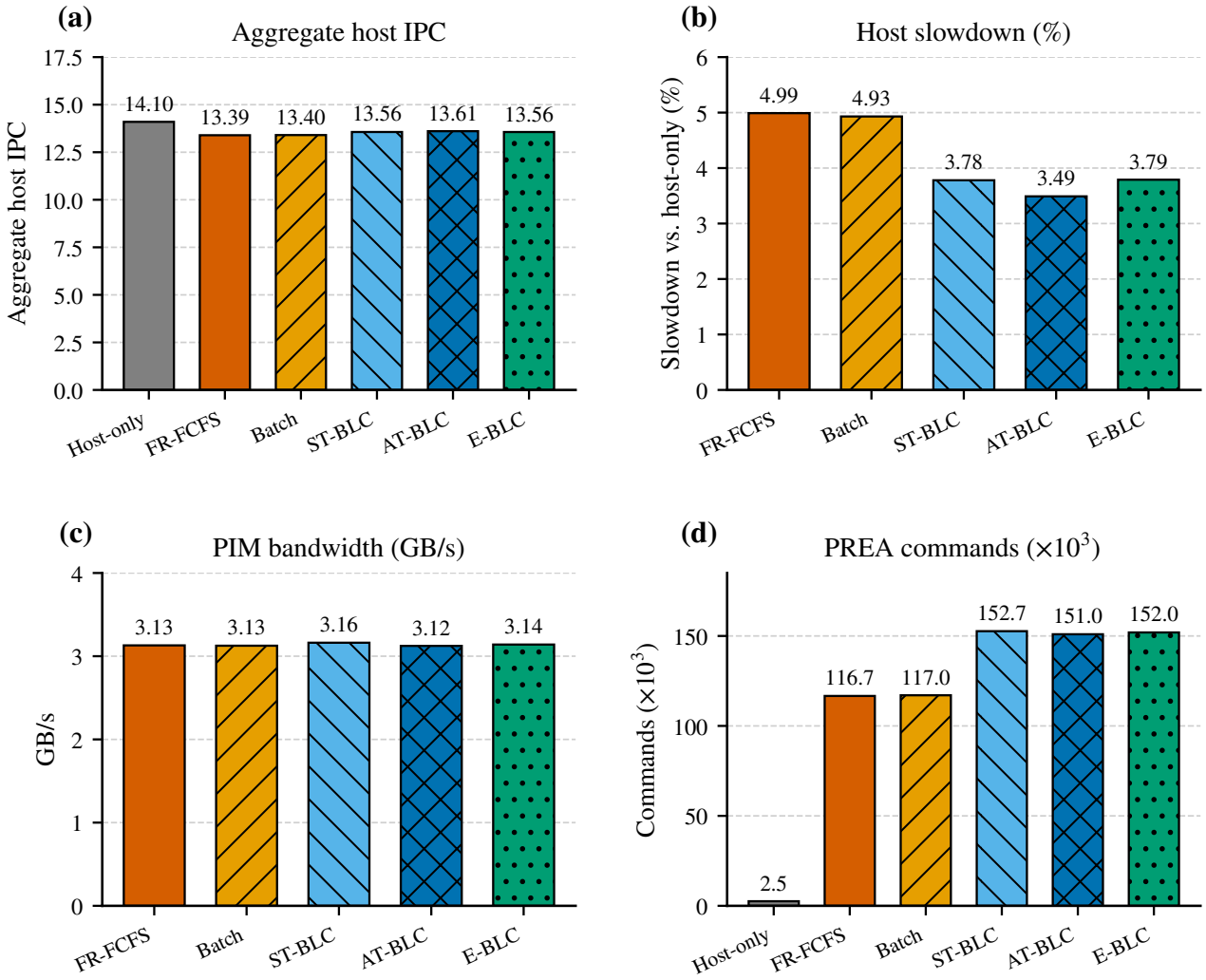


Fig. 11: Full-system evaluation on 16 out-of-order cores and 4-channel LPDDR5-6400: (a) aggregate host IPC, (b) host slowdown versus the host-only run, (c) PIM effective bandwidth, (d) rank-wide PREA commands.

VII. Conclusion and Future Work

We implemented the ComPASS protocol and scheduling ideas in two simulators and evaluated them end to end. At the protocol level, `PIM-ACT` is functionally exact and removes the 351 ns mode-switch round trip; speed-ups over the PIM region range from -2.75% to $+7.16\%$ and are smaller than the paper's except for GEMV without AAO. At the system level, a dedicated PIM queue with turn-based balancing reduces host slowdown by up to 30% relative to FR-FCFS while keeping PIM bandwidth constant, at the price of about 30% more rank-wide precharges. Our proposed E-BLC is on par with ST-BLC and behind AT-BLC.

Future work. (1) Make N and P configurable in the controller and redo the N:P, d and λ sweeps on the full-length configuration. (2) Spread host traffic over all four channels

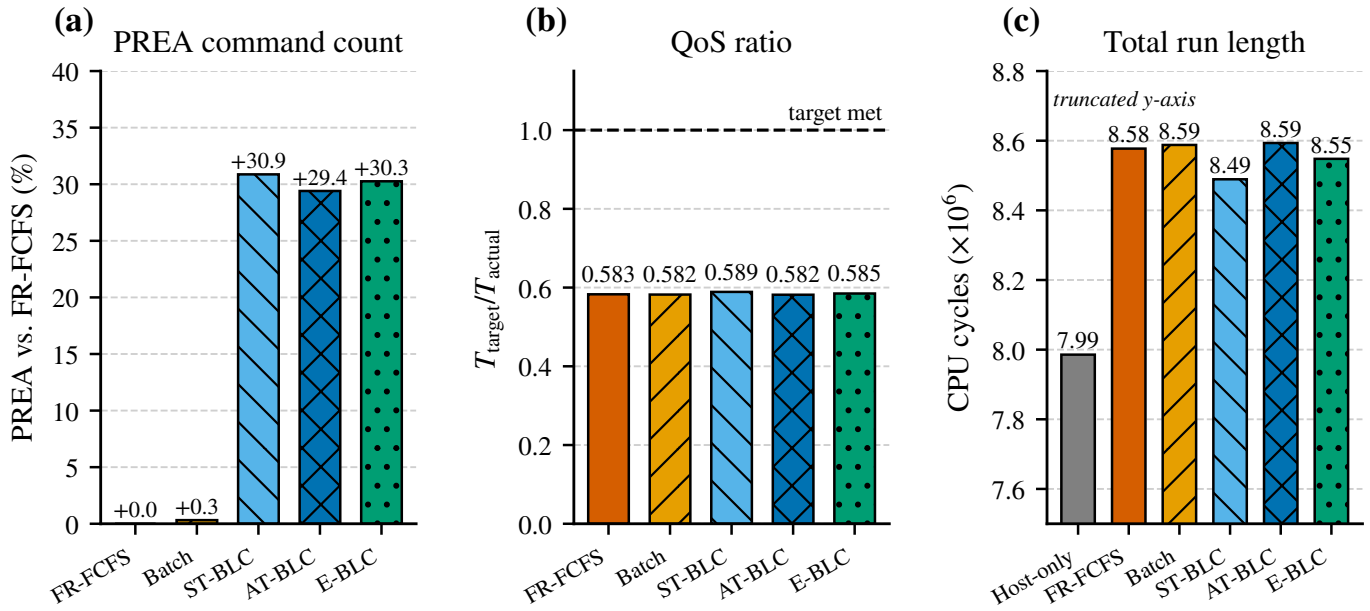


Fig. 12: (a) PREA commands relative to FR-FCFS (positive = more PREA), (b) QoS ratio $T_{\text{target}}/T_{\text{actual}}$ (dashed line: target met), (c) total run length (y-axis truncated).

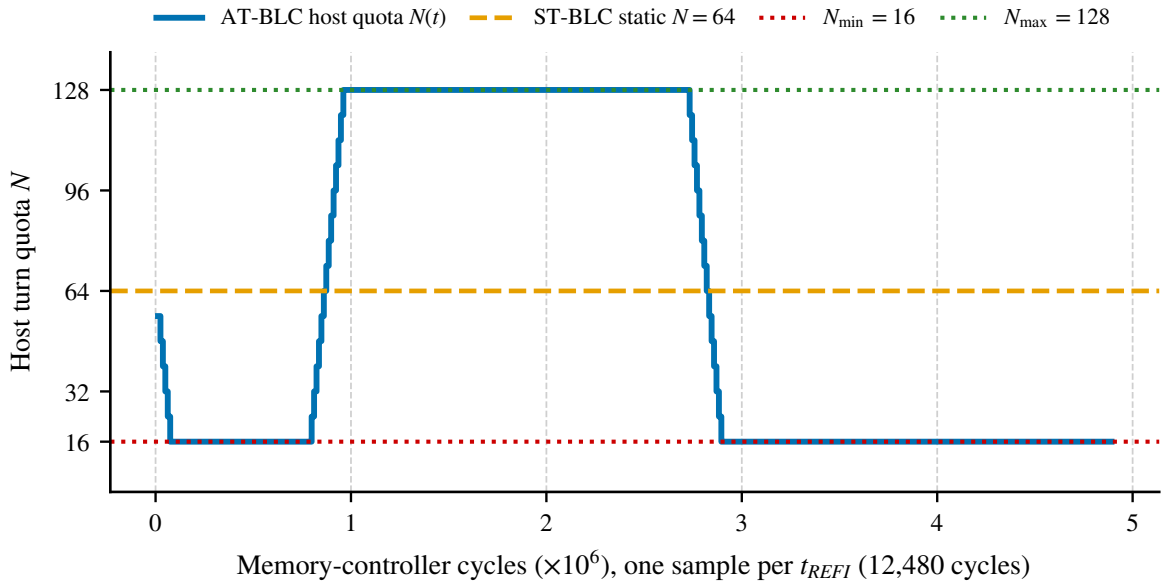


Fig. 13: AT-BLC host turn quota $N(t)$ (channel 0) with the static $N = 64$ of ST-BLC and the bounds $N_{\text{min}} = 16$, $N_{\text{max}} = 128$; one sample per t_{REFI}.

and use mixed SPEC/GAPBS workloads to give E-BLC a fair test. (3) Report PIM completion time against T directly instead of total cycles. (4) Calibrate AAO factors per optype and revisit ReLU readback, e.g. by closing PIM rows lazily before readback. (5) Repeat runs across trace mixes to obtain confidence intervals.

VIII. Reproducibility

Repository layout: `PIMSimulator/` (Part A) and `ramulator/` (Part B), plus `docs/deviations.md`. Requirements: Linux or WSL2, g++ with C++11, SCons, Make, Python 3.8+ with numpy and matplotlib. The figures in this report were regenerated as vector PDFs from the CSVs and logs above; the TikZ diagrams are embedded in the LaTeX source.

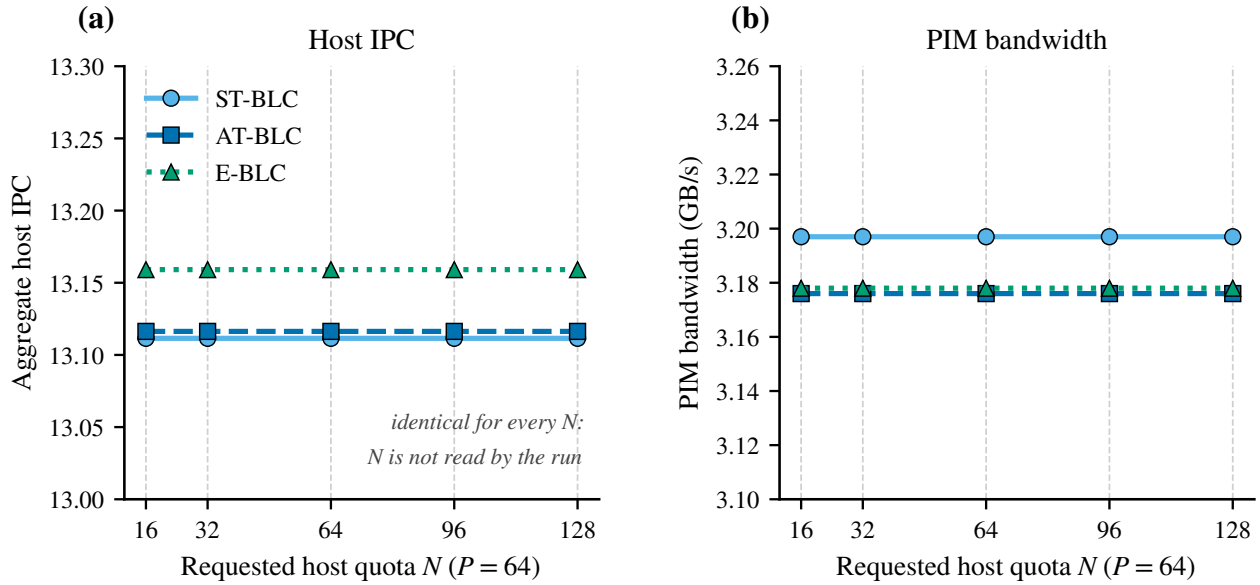


Fig. 14: N:P sweep (shortened configuration). Flat lines show that the requested N had no effect on the run.

TABLE IX: Reproduction status of the main claims.

Claim (paper)	**Our measurement**	**Status**
Mode-switch round trip ≈350 ns	351–352 ns, six phases measured	Reproduced
`PIM-ACT` removes mode switching	4,864 packets eliminated, bit-exact results	Reproduced
GEMV gain, AAO off +2.54%	+3.25% (region), +0.62% (whole run)	Reproduced over region
GEMV gain, AAO on +9.65%	+6.32% (region), +1.28% (whole run)	Partly (AAO factors assumed)
ADD/MUL gains ≈10–11%	+7.16% / +2.36% (region)	Not reproduced
ReLU gain +4.5%	−2.75%	Not reproduced
Balancers bound host slowdown	4.99% → 3.49–3.78% at equal PIM BW	Reproduced qualitatively
PREA latency hidden	PREA count +30%, host slowdown lower	Partly (count not reduced)
Meets PIM QoS target	ratio 0.58–0.59 for all policies	Not shown
N:P / λ sensitivity	sweep inconclusive	Not reproduced

TABLE X: Known deviations from the paper and limitations of our evaluation.

Item	**Description and impact**
Optype numbering	Transfers numbered 32–36 instead of 32 and 36–39; cosmetic, no timing effect.
AAO timing factors	0.5/0.6/0.6/0.5 of $t_{RCD}/t_{RAS}/t_{RC}/t_{RP}$ are our assumption (the paper gives no cycle counts); the main driver of the GEMV-with-AAO shortfall and of AAO having no effect on ADD/MUL/ReLU.
λ inconsistency	Paper text gives $\lambda = 0.01$ in one place and 0.1% in another; we use 0.01 as a configurable parameter. No λ sweep was completed.
Benchmark sizing	ADD, MUL, ReLU laid out as 2-D arrays (1024 × 1024, 2048 × 1024, 4096 × 1024); matched-size baselines added so GEMV compares 4096 × 4096 against 4096 × 4096.
Workloads	SPEC CPU2006 traces and a synthetic GEMV micro-request stream, versus SPEC CPU2017/GAPBS and GPT-3 GEMV shapes in the paper. LPDDR5-PIM compute units are not modelled internally.
Host channel usage	Host requests reach channels 0 and 2 only (Figure 7c); per-channel balancing and E-BLC's BLP opportunities are limited.
Statistical robustness	One deterministic run per configuration; no variation over trace mixes. Differences among ST-BLC/AT-BLC/E-BLC (0.3 pp) are small and untested for robustness.
QoS metric	Uses total simulated cycles, not PIM completion time; all policies < 1.
Clock ratio	Config sets `cpu_tick=mem_tick=1` and bandwidth uses 3.2 GHz, while the spec document mentions a 4 GHz CPU; we follow the code.
Sweep defect	Requested N is not applied (Section V-G).
Superseded artifacts	`evaluate_fig7.py`, `aao_speedup_data.csv` and `fig7_aao_speedup.png` used host wall-clock time and mismatched sizes; we regenerated Figure 8 from the cycle matrix.

IX. COMPASS+: IMPROVEMENTS AND QUANTITATIVE COMPARISON

Sections I–VIII are the paper-faithful reproduction. This section reports the further improvements our team made to

TABLE XI: Commands to reproduce the experiments.

Step	Command
Part A data	`python3 data/generate_all_data.py` (seed 1113)
Part A build	`scons NO_LIBRARY=1 -j$(nproc)`
Part A matrix	`./run_step3_matrix.sh | tee results/step3_matrix/final.txt`
Legacy precharge	`PIMACT_PREA_AWARE=0 ./run_step3_matrix.sh`
Mode-switch trace	`PIM_TRACE_MODE_SWITCH=1 ./sim -gtest_filter="PIMKernelFixture.*"` then `scripts/analyze_mode_switch.py`
Part B build	`make -j$(nproc)`
Part B runs	`python3 evaluate_policies.py` (writes `results/*.csv`)
Sweep	`python3 parameter_sweep.py` (see Section V-G)

TABLE XII: Limitations identified in the reproduced ComPASS controller.

ID	Limitation	Consequence
L1	Fixed adaptation step $d = 8$	Slow response to sudden host bursts; head-of-line blocking in the host queue
L2	Static margin $T' = 0.9\,T$	Withholds 10% of the time budget from the host even when PIM is far ahead
L3	PIM banks visited $0 \to 1 \to 2 \to 3$	Consecutive accesses stay in one LPDDR5 bank group and pay the longer same-group spacings (tRRD-L, tCCD-L, tWTR-L)
L4	AT-BLC epochs run during CPU warm-up; rate divided by total cycles	PIM disabled during warm-up gives $\delta = -1$, so N is pushed to $N_{\min}$; warm-up cycles permanently deflate the measured rate
L5	Turn state machine keeps cycling after PIM finishes	Arbitration bubbles and a clamped N for the rest of the run
L6	Opportunistic fill requests were charged to the other stream's turn counter	Incoming stream starts with a depleted quota and is evicted early
L7	`stoul` for trace addresses	32-bit overflow on Windows/MinGW for 48-bit addresses (crash)

the Ramulator controller, which we call **ComPASS+**, and compares them quantitatively. The figures come from the team's evaluation matrix of the modified simulator (same 16-core, 4-channel LPDDR5-6400 system and 262,144-micro-op GEMV stream as Table II). Because the modified code also changes shared infrastructure (bank mapping, trace parsing, turn accounting), *every* policy was re-run; comparisons must therefore be read *within* that matrix and not mixed with Table VIII (see Section IX-F).

A. Limitations Found in the Reproduced Controller

Debugging the reproduced controller exposed seven weaknesses (Table XII).

B. Improvements

Each limitation is addressed by one change inside the controller arbiter or trace parser (Table XIII). With W_{done} the PIM micro-ops completed and $\delta = (R_{\text{cur}} - R_{\text{tgt}})/R_{\text{tgt}}$ the relative progress error, the adaptive law becomes

$$s = \min\big(32,\ \max(4,\ \lfloor 100\,|\delta| \rfloor)\big), \tag{3}$$

$$R_{\text{cur}} = W_{\text{done}}/(clk - pim_start_clk), \tag{4}$$

$$N \leftarrow \begin{cases} \min(N_{\max}, N + s) & \delta > \lambda \ \wedge\ U_{\text{host}} > 0.5, \\ \max(N_{\min}, N - s) & \delta < -\lambda, \end{cases} \tag{5}$$

$$T' = \begin{cases} 0.95\,T & \delta > 0.05, \\ 0.90\,T & \delta < -\lambda \text{ (snap back)}, \end{cases} \tag{6}$$

and the PIM bank sequence is $b = (i \bmod 4) \cdot 4 + \lfloor i/4 \rfloor$, i.e. banks $0, 4, 8, 12, 1, 5, \ldots, 15$, which alternates bank groups on every activation.

C. Quantitative Results

Table XIV gives the full matrix. *ComPASS+ Step 1* is an intermediate configuration and *Final* the complete set; the team's notes do not itemise which of I1–I7 belong to Step 1, so only the Step 1→Final difference, not per-feature gains, can be quantified.

Table XV and Figure 16 quantify the Final configuration against every reference in the same matrix.

a) Summary of the quantities.:

- Slowdown $3.79\% \to 3.44\%$ against the in-matrix AT-BLC baseline (-0.35 pp, -9.2% relative; the team's notes round this to "about 10%"); IPC $13.5627 \to 13.6115$ ($+0.36\%$); weighted speedup $0.9530 \to 0.9572$.
- Against FR-FCFS the host regains 1.21% IPC and slowdown falls by 25%.
- PIM bandwidth is unchanged at 3.12–3.15 GB/s and all 65,536 micro-ops per channel complete.
- PREA commands rise by 3.5% over AT-BLC and by 29.6% over FR-FCFS.

Figure 17 shows $N(t)$ in the modified controller. The trace begins only after warm-up (≈0.6 M cycles, I5), enters near 96 and saturates at $N_{\max} = 128$ almost immediately, where it stays. Compared with Figure 13, the early plunge to $N_{\min}$ is gone. Note that after PIM completes (about 0.8 M cycles per the team's logs) I4 pins $N = N_{\max}$ by design, so the plateau over most of the run reflects surrender rather than ongoing adaptation.

TABLE XIII: Improvements implemented in ComPASS+ and the limitation each addresses.

ID	Improvement	Mechanism	File	Fixes / inspired by
I1	Proportional step	Step $s = \mathrm{clip}(\lfloor 100\lvert\delta\rvert \rfloor, 4, 32)$ replaces fixed $d = 8$; large leads move N faster, small leads gently	`Controller.h`	L1; PFM-style control
I2	Adaptive safety margin	T' relaxes from $0.90T$ to $0.95T$ when PIM is $> 5\%$ ahead; snaps back to $0.90T$ the moment $\delta < -\lambda$	`Controller.h`	L2
I3	Bank-group stride interleaving	PIM generator alternates bank groups (0,4,8,12,1,5,…); each row is still read in full (64 columns), so row hits are kept	`Controller.h`	L3; HetMap, OptiPIM
I4	Post-completion turn surrender	When $W_{\mathrm{done}} \geq W_{\mathrm{total}}$: host turn permanently, $N = N_{\max}$, balancer bypassed	`Controller.h`	L5
I5	Warm-up sync, active-PIM clock	AT-BLC epochs gated on `warmup_complete`; rate uses $clk - pim_start_clk$	`Controller.h`	L4
I6	Zero-leak turn accounting	Opportunistic fill requests are served without incrementing the other stream's consecutive counter	`Controller.h`	L6
I7	64-bit trace parsing	`stoul`→`stoull` in all three trace readers	`Processor.cpp`	L7
I8	Non-destructive plotting	`plot_results.py <dir>` writes to a chosen folder; earlier plots kept in `results/previous_plots/`	`plot_results.py`	workflow

TABLE XIV: Evaluation matrix of the modified simulator (16 cores + 262,144 PIM micro-ops). Slowdown is relative to the host-only IPC of 14.0967; bold marks the best co-execution value per column.

Policy	Host IPC	Slowdown (%)	Weighted speedup	PIM BW (GB/s)	PIM speedup	PREA ($\times 10^3$)	QoS ratio
Host-only	14.0967	0 (ref.)	1.0000	—	—	2.5	—
FR-FCFS	13.4485	4.60	0.9544	3.12	1.00	**120.7**	0.581
Batch (PAR-BS)	13.3854	5.05	0.9468	**3.15**	1.01	119.7	**0.586**
ST-BLC	13.5535	3.85	0.9557	3.14	1.01	151.1	0.585
AT-BLC (baseline)	13.5627	3.79	0.9530	3.13	1.00	151.1	0.583
ComPASS+ Step 1	13.6015	3.51	0.9557	**3.15**	1.01	154.7	**0.586**
ComPASS+ Final	**13.6115**	**3.44**	**0.9572**	3.14	1.01	156.4	0.585
E-BLC (ours, Section V)	13.5729	3.72	0.9538	3.14	1.01	151.7	0.585

TABLE XV: ComPASS+ Final versus each reference in the same matrix (IPC 13.6115, slowdown 3.44%).

Reference	IPC gain (%)	Slowdown (pp)	Rel. slowdown (%)	PREA (%)
FR-FCFS	+1.21	−1.16	−25.2	+29.6
Batch	+1.69	−1.61	−31.9	+30.7
ST-BLC	+0.43	−0.41	−10.6	+3.5
AT-BLC (base)	+0.36	−0.35	−9.2	+3.5
ComPASS+ Step 1	+0.07	−0.07	−2.0	+1.1
E-BLC	+0.28	−0.28	−7.5	+3.1

D. Trade-off Analysis

a) PREA count (+3.5%).: The extra PREAs (151,077 → 156,418) follow from longer host bursts: with N reaching 128 instead of sitting at 16, each return to PIM needs a rank-wide precharge, but each PREA now amortises up to 128 host accesses. The net effect is higher IPC, so the added precharges are hidden rather than harmful, in line with ComPASS's goal of overlapping turnaround latency.

b) Safety margin (10% → 5%).: Relaxation is conditional and reversible: it applies only while PIM is $> 5\%$ ahead and reverts in one step if $\delta < -\lambda$. In the team's runs PIM finished in about 0.8 M cycles against $T = 5$ M with no deadline miss.

c) PIM bandwidth and hardware cost.: Bandwidth is preserved (3.13→3.14 GB/s). All changes are integer comparisons, clamps and a bank-index permutation inside the arbiter; no new DRAM pins, commands or tables are required.

E. Design Rationale

Proportional, not maximal, steps. Jumping straight to $N_{\max}$ would let a host surge stall PIM in the next epoch and swing N back to $N_{\min}$ (chattering). The proportional step, the $\lambda = 0.01$ dead band, the 50% host-utilisation gate, the $[16, 128]$ clamp and once-per-t_{REFI} evaluation damp this.

Bank-group striding keeps row locality. A macro-request still consumes a whole 2 KB row (64 columns) per activation, so the 98% row-hit rate is unaffected; only the *next* activation moves to a different bank group, which spreads PIM across independent groups and lowers collisions with host traffic.

Warm-up lock. During warm-up PIM was disabled, so the rate was zero and $\delta = -1$; dividing by total cycles then kept the rate low even after PIM started. Gating on `warmup_complete` and using active-PIM cycles removes this.

Surrender and zero-leak accounting. Without I4 the turn machine inserts periodic one-cycle bubbles after PIM ends; without I6 an opportunistic fill shortens the next owner's quota.

64-bit parsing. Linux (LP64) has 64-bit `unsigned long`; Windows (LLP64) has 32 bits, so 48-bit addresses overflowed. `stoull` also avoids silent truncation of address bits 32–33 in the 34-bit LPDDR5 map.

F. Critical Assessment and Caveats

We report these honestly because they bound what the numbers show.

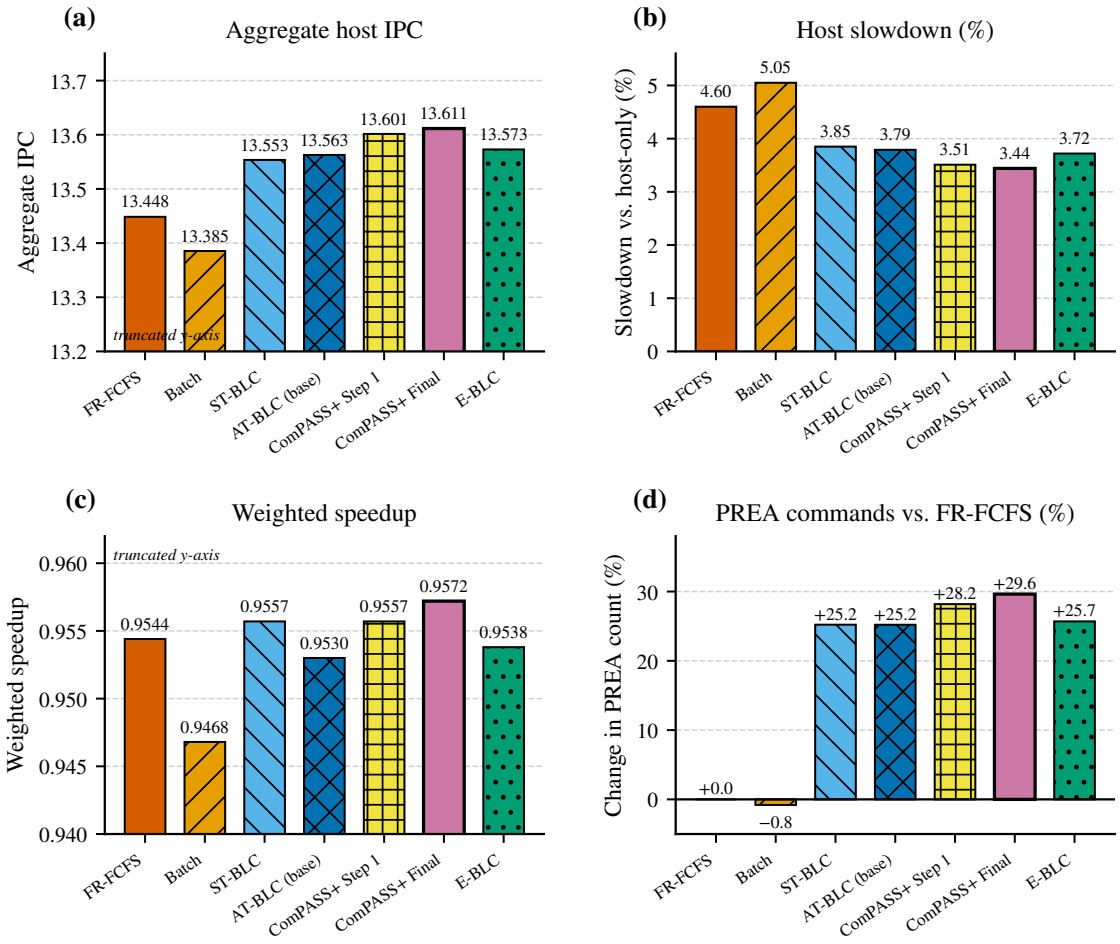


Fig. 15: Modified-simulator results: (a) aggregate host IPC, (b) host slowdown, (c) weighted speedup (y-axes truncated in a, c), (d) change in PREA count relative to FR-FCFS (positive = more PREA). The ComPASS+ Final bar is outlined.

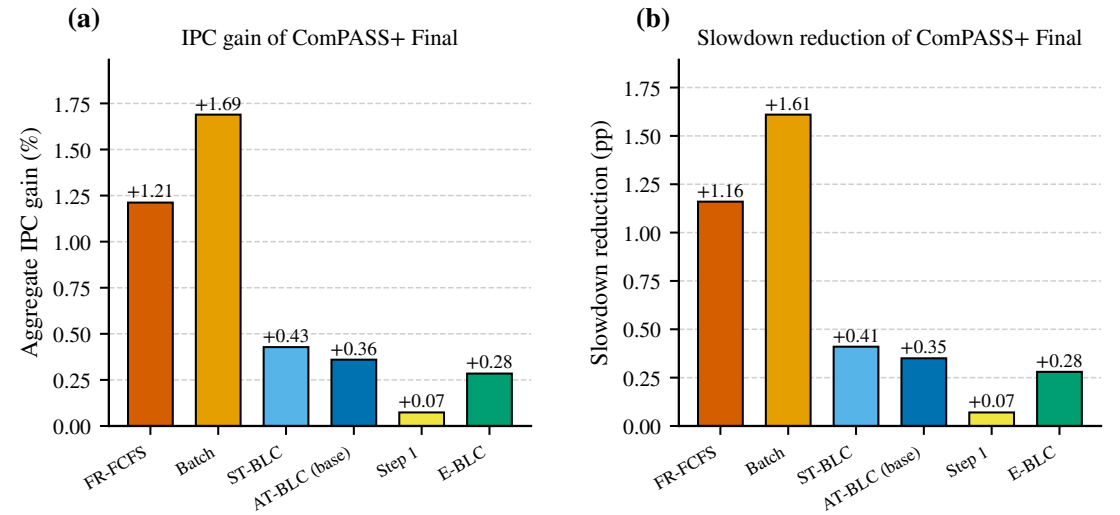


Fig. 16: Gain of ComPASS+ Final over each reference policy: (a) aggregate IPC gain, (b) reduction in host slowdown in percentage points.

1) **Cross-matrix drift.** Policies that ComPASS+ does not target also moved between our reproduction matrix (Table VIII)

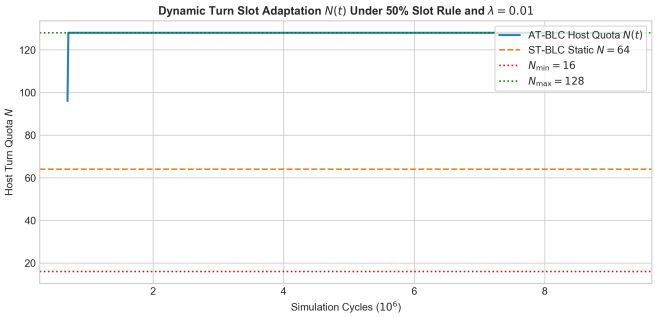


Fig. 17: Host quota $N(t)$ under ComPASS+ (original plot from the team's run). Compare Figure 13.

TABLE XVI: Host slowdown (%) of the same policies in the reproduction matrix (Table VIII) and in the modified-simulator matrix (Table XIV).

Policy	Reproduction	Modified	Shift (pp)
FR-FCFS	4.99	4.60	−0.39
Batch	4.93	5.05	+0.12
ST-BLC	3.78	3.85	+0.07
AT-BLC	3.49	3.79	+0.30
E-BLC	3.79	3.72	−0.07
ComPASS+ Final	—	3.44	—

and the modified one (Table XVI): FR-FCFS improved by 0.39 pp, AT-BLC worsened by 0.30 pp. The 0.35 pp gain over the in-matrix AT-BLC is of the same size, so with a single deterministic run per configuration it cannot be cleanly separated from configuration effects.

2) **Smaller gain against our own AT-BLC.** Against the AT-BLC of Table VIII (13.6055, 3.49%) the Final configuration gains only +0.04% IPC and −0.05 pp slowdown. The headline ≈10% relative gain depends on the weaker in-matrix baseline.
3) **No ablation.** Only Step 1 → Final (−0.07 pp) is separable; the individual contribution of I1–I7 is unmeasured.
4) **Loose QoS target.** PIM completes in about 16% of T, so "zero violations" is easily met; the PIM-bandwidth metric divides by total run length and is therefore insensitive to scheduling (3.12–3.15 GB/s for all policies).
5) **Unverified code.** The attached `Controller.cpp` is the upstream Ramulator file (SALP/ALDRAM/TLDRAM specialisations) and does not contain the ComPASS+ changes, which live in `Controller.h` and `Processor.cpp`; the mechanisms above are as documented by the team.
6) **Magnitude.** The −0.35 pp slowdown is small in absolute terms, and PREA commands increase.

a) Conclusion of this section.: ComPASS+ fixes real defects (warm-up lock, turn-accounting leak, 64-bit parsing) and gives the lowest slowdown (3.44%) and highest IPC and weighted speedup of all policies in its matrix at unchanged PIM bandwidth. The margin over the best earlier policy is small (about 0.05–0.35 pp depending on the baseline), so repeated runs over more trace mixes and a per-feature ablation are the next step before claiming a robust gain.

TABLE XVII: Per-channel statistics (PIM requests are 65,536 per channel in every run). Host traffic appears only on channels 0 and 2.

Policy	Ch	Host req.	PREA	PIM BW (GB/s)	PIM hit (%)
Host-only	0	534,416	1,277	—	—
	2	537,011	1,270	—	—
FR-FCFS	0	538,728	59,808	0.782	98.23
	1	0	367	0.782	98.44
	2	541,635	56,110	0.782	98.18
	3	0	367	0.782	98.44
Batch	0	539,620	56,920	0.781	98.19
	1	0	370	0.781	98.44
	2	541,968	59,365	0.781	98.21
	3	0	370	0.781	98.44
ST-BLC	0	538,872	74,866	0.791	98.29
	1	0	356	0.791	98.44
	2	541,678	77,087	0.791	98.25
	3	0	356	0.791	98.44
AT-BLC	0	541,666	75,036	0.781	98.26
	1	0	356	0.781	98.44
	2	544,456	75,204	0.781	98.25
	3	0	356	0.781	98.44
E-BLC	0	538,941	75,168	0.785	98.27
	1	0	356	0.785	98.44
	2	541,668	76,071	0.785	98.26
	3	0	356	0.785	98.44

APPENDIX A
PER-CHANNEL BREAKDOWN

Table XVII lists the per-channel statistics behind Figure 7.

APPENDIX B
FILES CREATED OR MODIFIED

Table XVIII lists the principal source files.

REFERENCES

[1] S. Yu, H. Kim, K. Jeun, S. Hwang, S. Cho, and E. Lee, "ComPASS: A Compatible PIM Protocol Architecture and Scheduling Solution for Processor-PIM Collaboration," in *Proc. 58th IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, Seoul, 2025. doi:10.1145/3725843.3756017.
[2] S. Lee *et al.*, "Hardware Architecture and Software Stack for PIM Based on Commercial DRAM Technology," in *Proc. ISCA*, 2021.
[3] M. He *et al.*, "Newton: A DRAM-maker's Accelerator-in-Memory (AiM) Architecture for Machine Learning," in *Proc. MICRO*, 2020.
[4] F. Devaux, "The True Processing In Memory Accelerator," in *Hot Chips 31*, 2019.
[5] Y. Kim, W. Yang, and O. Mutlu, "Ramulator: A Fast and Extensible DRAM Simulator," *IEEE Computer Architecture Letters*, vol. 15, no. 1, pp. 45–49, 2016.
[6] P. Rosenfeld, E. Cooper-Balis, and B. Jacob, "DRAMSim2: A Cycle Accurate Memory System Simulator," *IEEE Computer Architecture Letters*, vol. 10, no. 1, pp. 16–19, 2011.
[7] S. Rixner, W. J. Dally, U. J. Kapasi, P. Mattson, and J. D. Owens, "Memory Access Scheduling," in *Proc. ISCA*, 2000.
[8] O. Mutlu and T. Moscibroda, "Parallelism-Aware Batch Scheduling: Enhancing both Performance and Fairness of Shared DRAM Systems," in *Proc. ISCA*, 2008.
[9] JEDEC, *Low Power Double Data Rate 5 (LPDDR5)*, JESD209-5.
[10] J. L. Henning, "SPEC CPU2006 Benchmark Descriptions," *ACM SIGARCH Computer Architecture News*, vol. 34, no. 4, 2006.
[11] Samsung SAIT, *PIMSimulator*, https://github.com/SAITPublic/PIMSimulator.

TABLE XVIII: Principal source files (full manifest in PROJECT_PROGRESS_AND_TODO.txt).

File	Change
PIMSimulator	
BusPacket.h/.cpp	PIM_ACT packet, isPIM, optype; constructor bug fix
PIMCmd.h	35-optype ISA, NO_OPTYPE sentinel
PIMRank.h/.cpp	doPIMACT: multi-bank SIMD FP16 execution
PimActTiming.h	AAO timing, write-delay calculation, runtime knobs
CommandQueue.cpp	all-bank-idle check, PREA-aware scan
MemoryController.cpp, Rank.cpp	rank-wide precharge, [PIM-REGION]/[MODE-SWITCH] instrumentation
tests/KernelTestCases.cpp, PIMKernel.cpp	four kernels, A3 bitwise verifier
scripts/analyze_mode_switch.py	mode-switch phase analysis
Ramulator	
LPDDR5.h/.cpp, configs/LPDDR5-PIM.cfg	LPDDR5-6400 model and configuration
Request.h, Memory.h	PIM request fields; pending-request accounting
Controller.h	pim_q, request generator, FR-FCFS, Batch, ST-BLC, AT-BLC, E-BLC, statistics
Processor.cpp	CRLF-tolerant trace parser
evaluate_policies.py, plot_results.py, parameter_sweep.py	experiment runner, plots, sweep